Full Stack Development with MERN

Project Documentation

DocSpot

Seamless Appointment Booking for Health

Field	Details
Team ID	LTVIP2026TMIDS69451
Date	21 February 2026
Stack	MongoDB · Express.js · React.js · Node.js

1. Introduction

Project Title: DocSpot: Seamless Appointment Booking for Health

Team Members & Roles

Member	Role	Responsibility
Member 1 POOJA	Full-Stack Developer	Backend APIs, MongoDB schemas, JWT Auth
Member 2 POOJA	Frontend Developer	React UI, Antd components, Axios integration
Member 3 POOJA	Full-Stack Developer	Doctor & Admin dashboards, File upload (Multer)
Member 4 POOJA	Testing & Documentation	UAT, API testing, project documentation

2. Project Overview

Purpose

DocSpot is a full-stack MERN web application that provides a digital bridge between patients and healthcare providers. The platform allows patients to discover verified doctors, book appointments online, upload medical documents, and track their appointment status — all from a single interface. The goal is to eliminate the inefficiencies of traditional, phone-based appointment booking systems.

Key Features

- Role-based access control for Patient, Doctor, and Admin users.
- Doctor discovery with profiles showing specialization, fees, experience, timings, and address.
- Online appointment booking form with date picker and medical document upload.
- Real-time appointment status tracking: Pending, Approved, Rejected, Cancelled.
- Admin approval workflow: doctors must be approved before appearing on the platform.
- In-app notification system for appointment status updates.
- JWT-based secure authentication with bcrypt password hashing.
- Responsive UI built with React.js, Ant Design (Antd), and Bootstrap.

3. Architecture

Frontend (React.js)

The frontend follows a component-based architecture using React.js. Pages are organized by user role — Patient, Doctor, and Admin — and communicate with the backend via Axios HTTP calls. Ant Design (Antd) and Bootstrap are used for UI components and responsive layout. React Router handles client-side navigation.

- Pages: Landing, Register, Login, Patient Dashboard, Doctor Dashboard, Admin Dashboard, Book Appointment, Appointment History.
- State management handled through React component state and props.
- API calls abstracted using Axios with JWT token headers for authenticated routes.

Backend (Node.js + Express.js)

The backend is a RESTful API server built with Node.js and Express.js. It is organized using the MVC pattern with separate route files for User, Doctor, and Admin modules. An authentication middleware (authMiddleware.js) validates JWT tokens on all protected routes.

- Authentication: jsonwebtoken for token generation and validation; bcryptjs for password hashing.
- File Handling: Multer middleware processes medical document uploads (multipart/form-data).
- Environment Config: dotenv manages sensitive variables (PORT, MongoDB URI, JWT_SECRET).
- Dev Tool: Nodemon for automatic server restarts during development.

Database (MongoDB + Mongoose)

MongoDB is used as the NoSQL database, managed through Mongoose ODM. Three primary collections store all application data:

Collection	Key Fields	Description
users	_id, name, email, password, phone, isDoctor, type, notification	Stores all registered users (patients & doctors)
doctors	_id, userID (FK), fullname, email, specialization, experience, fees, timings, address, status	Doctor profile data; userID links to users collection
appointments	_id, doctorInfo, userInfo, date, document, status	Booking records linking patients and doctors

4. Setup Instructions

Prerequisites

- Node.js (v18+) and npm — https://nodejs.org/en/download/
- MongoDB Community Edition — https://www.mongodb.com/try/download/community
- Git — https://git-scm.com/
- A code editor (VS Code recommended)

Installation

Step 1: Clone the repository

```
git clone https://github.com/[your-username]/docspot.git
```

Step 2: Install backend dependencies

```
cd docspot/backend  &&  npm install
```

Step 3: Install frontend dependencies

```
cd ../frontend  &&  npm install
```

Step 4: Configure environment variables

Create a .env file in the backend directory with the following variables:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/docspot
JWT_SECRET=your_jwt_secret_key
```

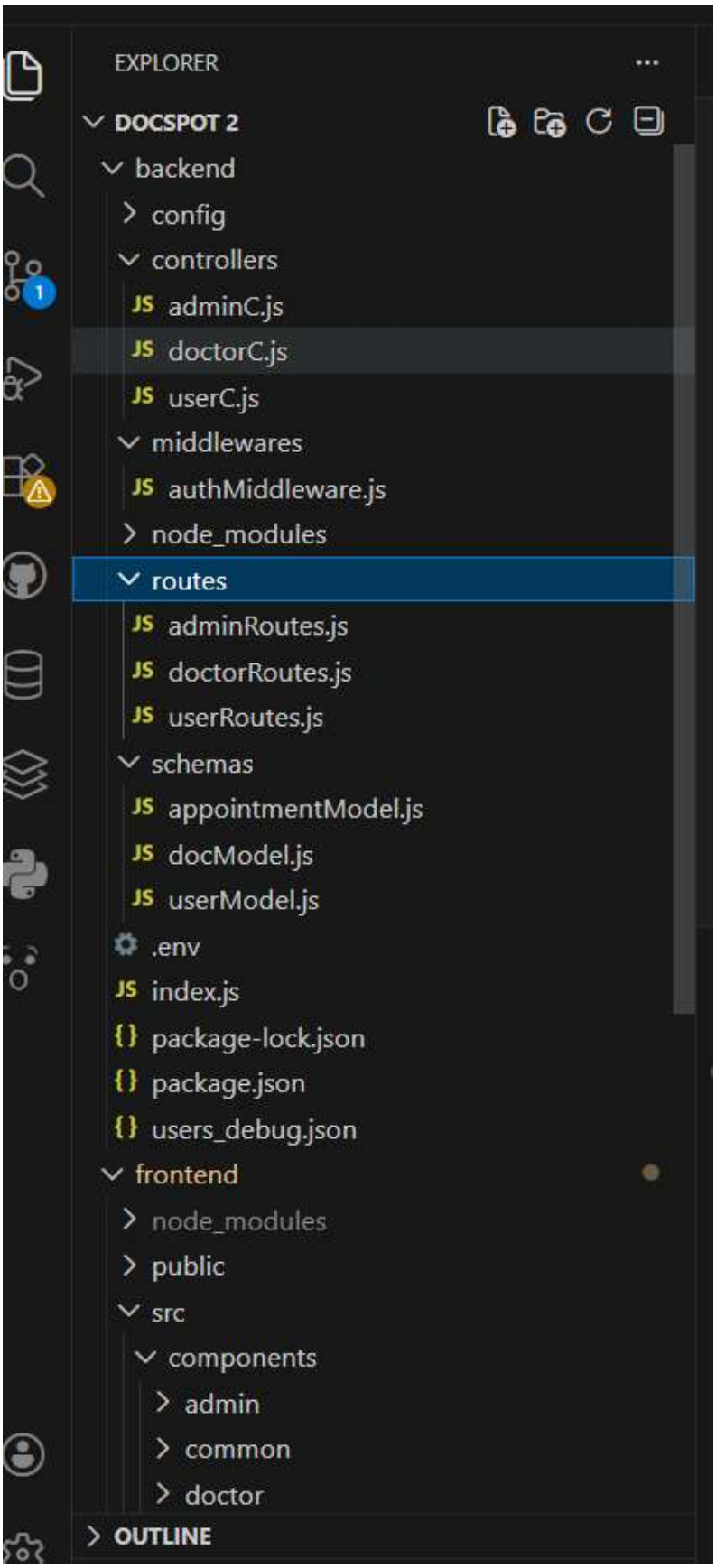
5. Folder Structure

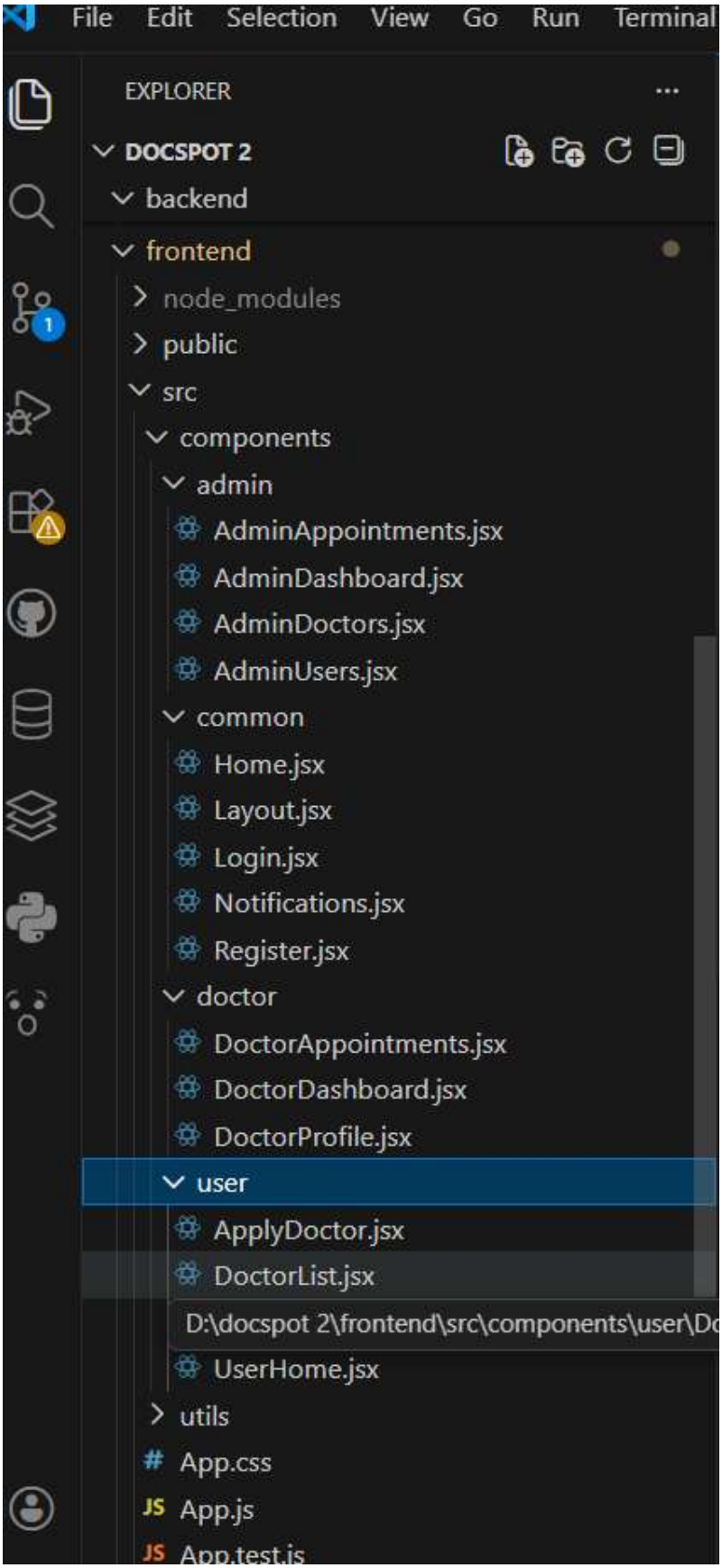
Client (Frontend)

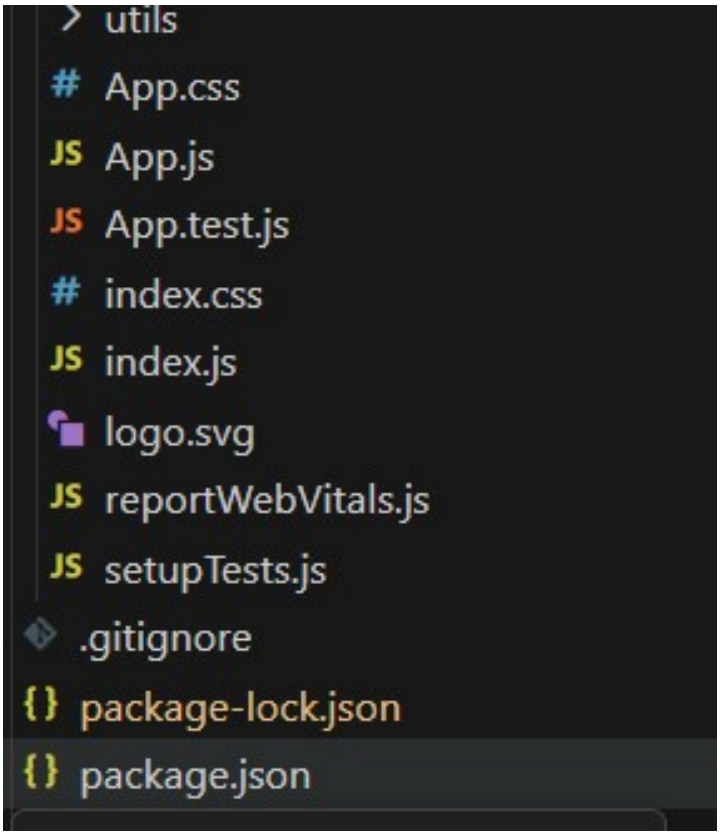
```
frontend/
  src/
    components/      # Reusable UI components
    pages/           # Role-based page components
      patient/       # Patient dashboard, booking, history
      doctor/        # Doctor dashboard, profile
      admin/         # Admin dashboard, user/doctor management
    App.js           # Root component with routing
    index.js         # React entry point
  package.json
```

Server (Backend)

```
backend/
  config/
    config.js        # MongoDB connection setup
  middleware/
    authMiddleware.js # JWT token verification
  models/
    userModel.js     # User schema (Mongoose)
    doctorModel.js   # Doctor schema (Mongoose)
    appointmentModel.js # Appointment schema
  routes/
    userRoutes.js    # Patient-facing endpoints
    doctorRoutes.js  # Doctor-facing endpoints
    adminRoutes.js   # Admin-facing endpoints
  index.js           # Express server entry point
  .env               # Environment variables
  package.json
```


EXPLORER
DOCSPOT 2
backend
config
controllers
adminC.js
doctorC.js
userC.js
middlewares
authMiddleware.js
node_modules
routes
adminRoutes.js
doctorRoutes.js
userRoutes.js
schemas
appointmentModel.js
docModel.js
userModel.js
.env
index.js
package-lock.json
package.json
users_debug.json
frontend
node_modules
public
src
components
admin
common
doctor
OUTLINE


File
Edit
Selection
View
Go
Run
Terminal
EXPLORER
DOCSPOT 2
backend
frontend
node_modules
public
src
components
admin
AdminAppointments.jsx
AdminDashboard.jsx
AdminDoctors.jsx
AdminUsers.jsx
common
Home.jsx
Layout.jsx
Login.jsx
Notifications.jsx
Register.jsx
doctor
DoctorAppointments.jsx
DoctorDashboard.jsx
DoctorProfile.jsx
user
ApplyDoctor.jsx
DoctorList.jsx
D:\docspot 2\frontend\src\components\user\Do
UserHome.jsx
utils
App.css
App.js
App.test.js


utils
App.css
App.js
App.test.js
index.css
index.js
logo.svg
reportWebVitals.js
setupTests.js
.gitignore
package-lock.json
package.json

6. Running the Application

Backend

Navigate to the backend directory and start the server:

```
cd backend
npm start        # or: nodemon index.js
# Server runs at http://localhost:5000
```

Frontend

In a separate terminal, navigate to the frontend directory and start the React app:

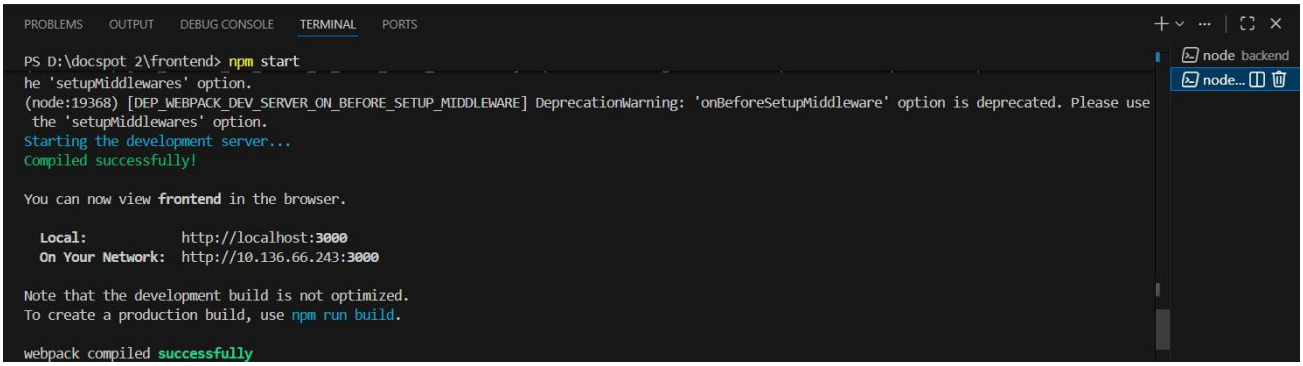
```
cd frontend
npm start        # React app runs at http://localhost:3000
```

The DocSpot application will be accessible at http://localhost:3000. Ensure MongoDB is running locally or your MongoDB Atlas connection string is correctly set in .env.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\docspot 2> cd backend
PS D:\docspot 2\backend> npm install

up to date, audited 153 packages in 2s

24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\docspot 2\backend> npm run dev

> docspot-backend@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.13
[nodemon] to restart at any time, enter `rs`
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\docspot 2\backend> npm install

found 0 vulnerabilities
PS D:\docspot 2\backend> npm run dev

> docspot-backend@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.13
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
🚀 Server running on http://localhost:5000
✅ MongoDB Connected: localhost
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\docspot 2> cd frontend
PS D:\docspot 2\frontend> npm install
npm warn ERESOLVE overriding peer dependency
npm warn While resolving: framer-motion@10.18.0
npm warn Found: react@19.2.4
npm warn node_modules/react
npm warn   react@"^19.2.4" from the root project
npm warn   70 more (@ant-design/cssinjs, @ant-design/cssinjs-utils, ...)
npm warn
npm warn Could not resolve dependency:
npm warn peerOptional react@"^18.0.0" from framer-motion@10.18.0
npm warn node_modules/mdb-react-ui-kit/node_modules/framer-motion
npm warn   framer-motion@"^10.16.4" from mdb-react-ui-kit@10.0.0
npm warn   node_modules/mdb-react-ui-kit
npm warn
npm warn Conflicting peer dependency: react@18.3.1
```



7. API Documentation

All API endpoints are prefixed with the base URL: http://localhost:5000/api. Protected routes require a valid JWT token in the request header: Authorization: Bearer <token>.

User Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/user/register	Register a new user	No
POST	/api/user/login	Login and receive JWT token	No
GET	/api/user/getalldoctors	Get list of all approved doctors	Yes
POST	/api/user/book-appointment	Book a new appointment	Yes
GET	/api/user/appointments	Get all appointments for logged-in user	Yes
POST	/api/user/cancel-appointment	Cancel a booked appointment	Yes
GET	/api/user/notifications	Get notifications for the	Yes

		user	

Doctor Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/doctor/apply-doctor	Submit doctor registration application	Yes
GET	/api/doctor/appointments	Get all appointments for the doctor	Yes
POST	/api/doctor/update-status	Approve or reject a patient appointment	Yes
GET	/api/doctor/profile	Get doctor profile details	Yes
POST	/api/doctor/update-profile	Update doctor profile info	Yes

Admin Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/admin/getallusers	Get all registered users	Yes (Admin)
GET	/api/admin/getalldoctors	Get all doctors and their status	Yes (Admin)
POST	/api/admin/approve-doctor	Approve a pending doctor application	Yes (Admin)
POST	/api/admin/reject-doctor	Reject a doctor application	Yes (Admin)

8. Authentication

DocSpot uses JWT (JSON Web Token) based authentication for all protected routes. Here is how the authentication flow works:

Registration & Login:

- On registration, the user's password is hashed using bcryptjs before being stored in MongoDB.
- On successful login, the server generates a signed JWT token using the JWT_SECRET from .env.
- The token is returned to the client and stored for subsequent API requests.

Token Usage:

- All protected API routes require the JWT token in the Authorization header: Bearer <token>.
- The authMiddleware.js verifies the token on each protected request and extracts user identity.
- If the token is invalid or expired, the server returns a 401 Unauthorized response.

Role-Based Access:

- The token payload includes the user's role (patient, doctor, admin), enabling role-based route protection.
- Admin routes are protected with additional checks to verify the isAdmin flag on the user record.

9. User Interface

The DocSpot UI is built with React.js and styled using Ant Design (Antd) and Bootstrap for a responsive, modern design. Below are the key screens:

Screen	Description
Landing Page	DocSpot branding, value proposition, Login and Register CTAs.
Register Page	Form to sign up as a Patient or Doctor with validation.
Login Page	Secure login with JWT-based session; redirects to role dashboard.
Patient Dashboard	Grid of all approved doctors with specialization, fees, and Book Now button.
Book Appointment	Date picker and document upload form to submit a booking request.
Appointment History	List of all bookings with real-time status badges and Cancel option.
Doctor Dashboard	Incoming appointment requests with Accept / Reject action buttons.
Admin Dashboard	Manage all users, approve/reject doctor applications, monitor appointments.

Note: Actual screenshots to be inserted above upon deployment.

10. Testing

Testing Strategy

DocSpot was tested using a combination of manual functional testing (UAT) and API-level testing to ensure all features work as expected across all user roles.

Tools Used

- Postman – API endpoint testing for all User, Doctor, and Admin routes.
- Browser DevTools – Frontend debugging and network request inspection.
- Manual UAT – Role-based scenario testing by team members simulating Patient, Doctor, and Admin workflows.

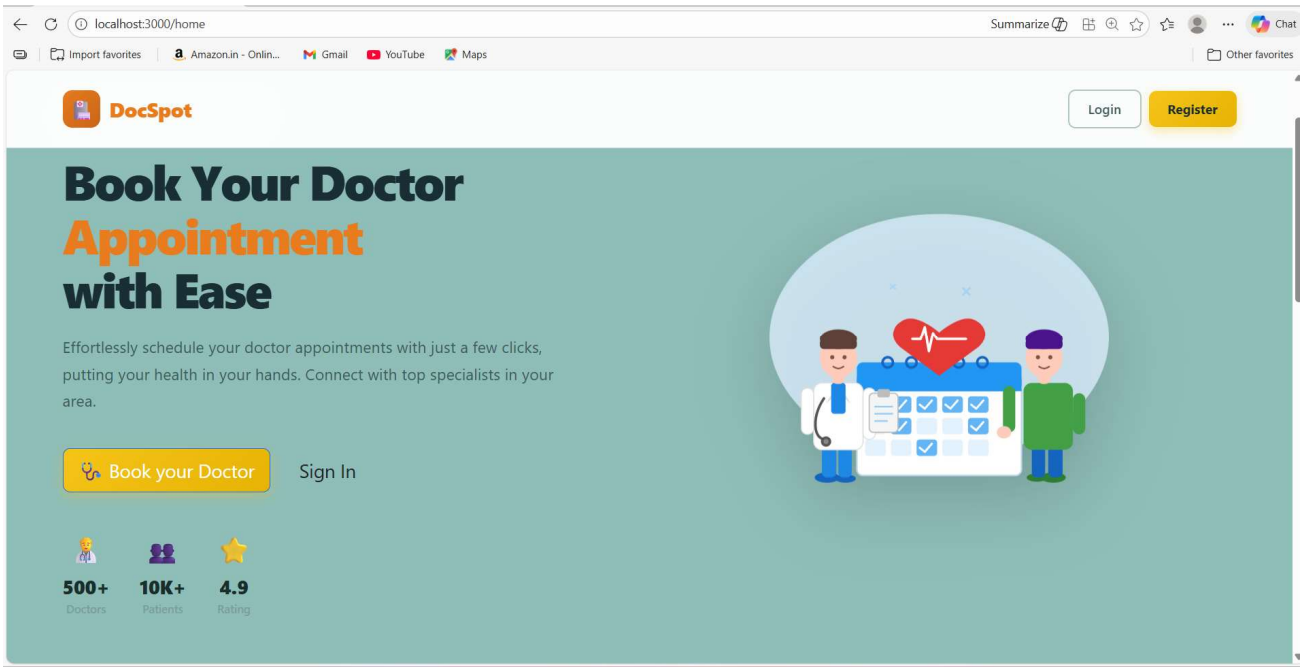
Test Summary

Module	Total	Passed	Failed	Pass Rate
User Registration & Login	8	8	0	100%
Doctor Listing & Search	6	6	0	100%
Appointment Booking	10	9	1	90%
Doctor & Admin Dashboards	14	14	0	100%
Notifications	4	3	1	75%
Security & Auth	5	5	0	100%
TOTAL	63	61	2	96.8%

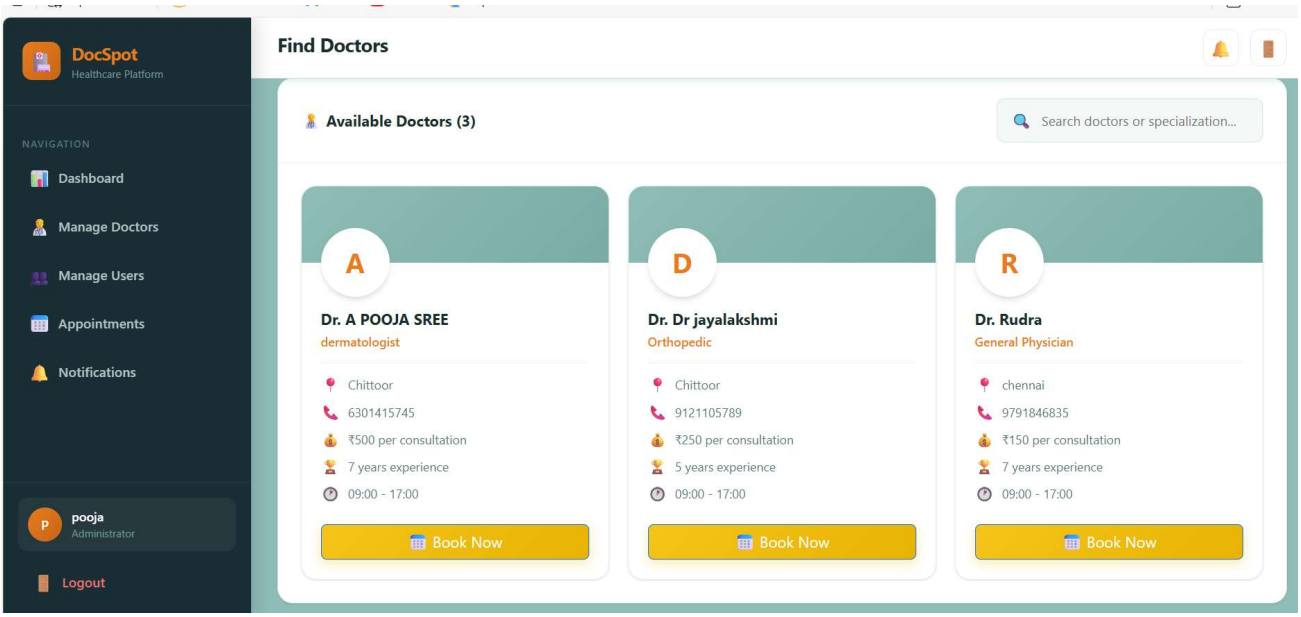
11. Screenshots or Demo

The following screenshots showcase the key screens of the DocSpot application:

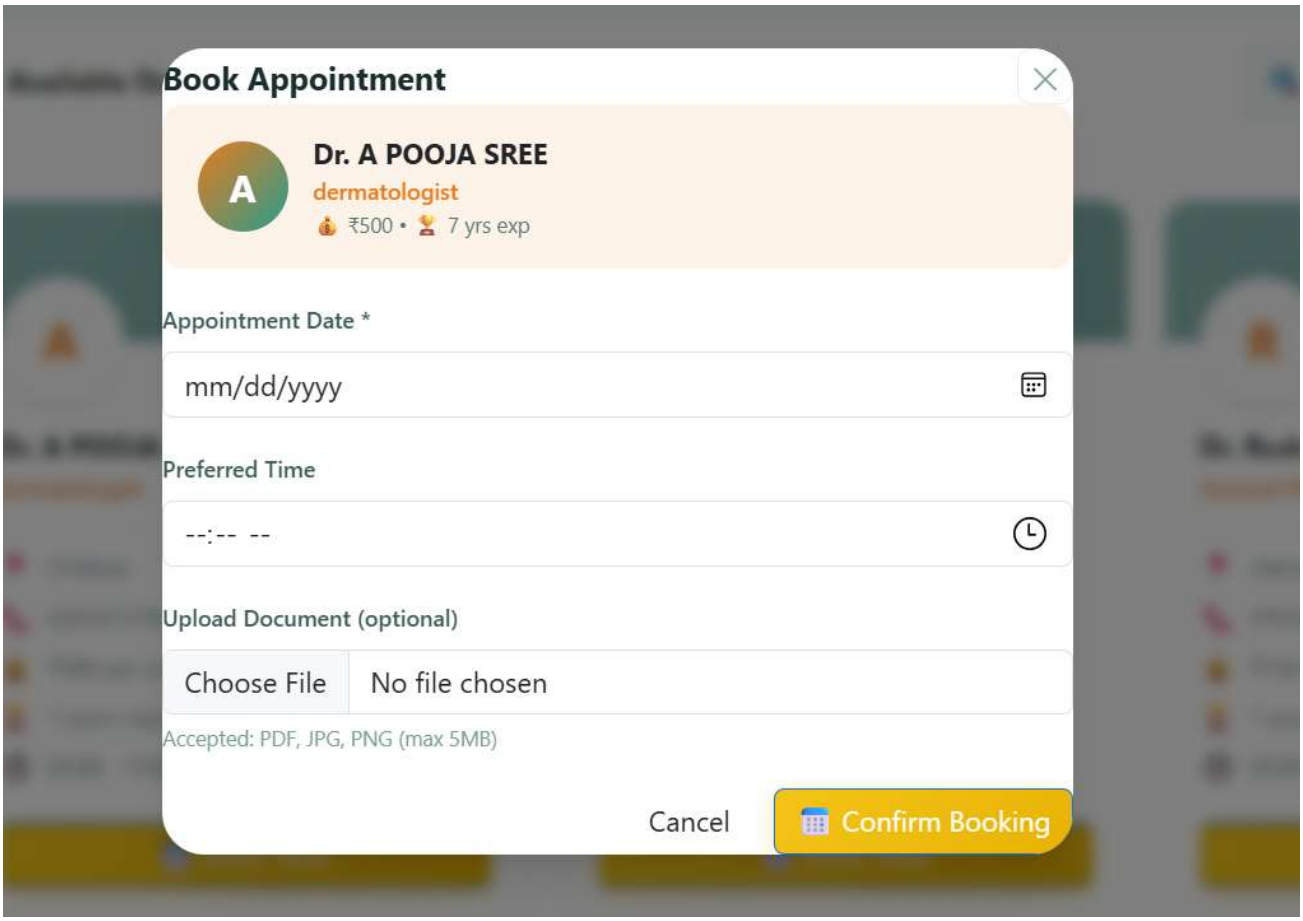
- Landing Page –



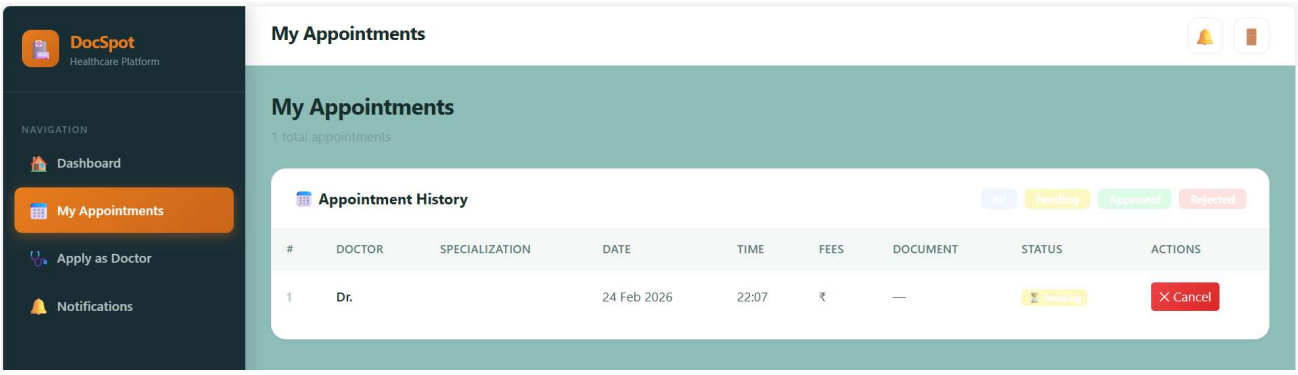
- Patient Dashboard showing doctor listings –



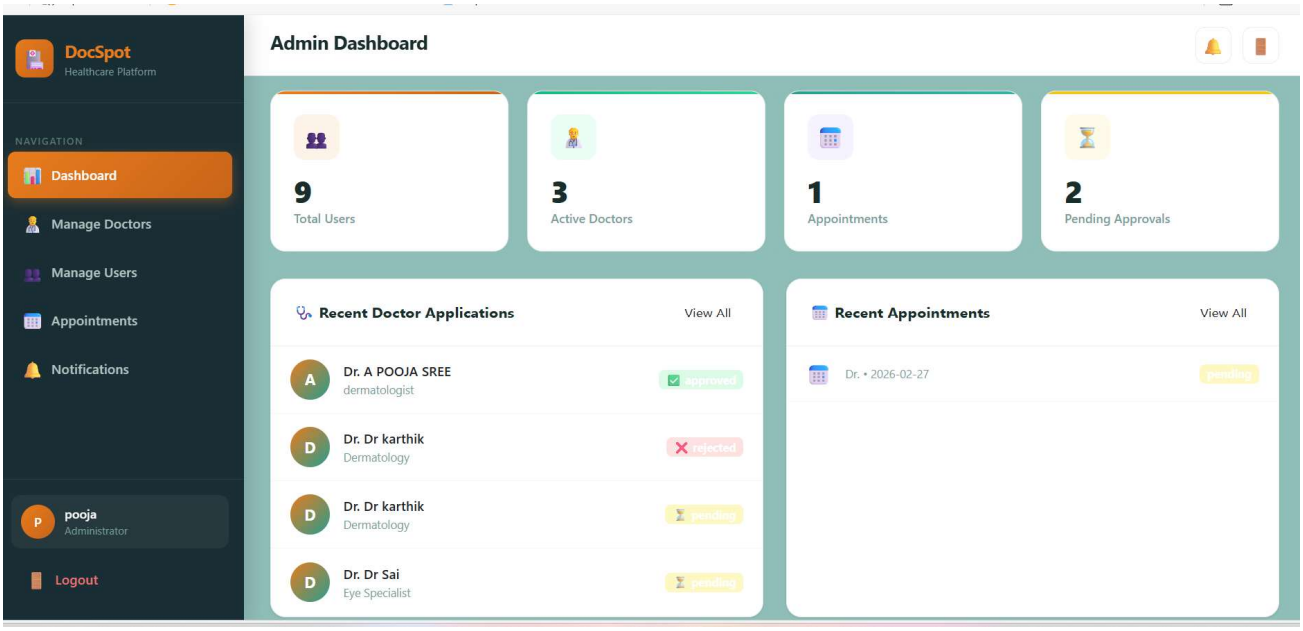
- Book Appointment form with date picker and document upload –



- Appointment History with status badges –



- Admin Dashboard –



- Doctor Approval Panel –

#	DOCTOR	SPECIALIZATION	EMAIL	PHONE	FEES	EXPERIENCE	STATUS	ACTIONS
1	Dr. A POOJA SREE	dermatologist		6301415745	₹500	7 yrs	approved	Reject
2	Dr. Dr karthik	Dermatology	pandupooja1277@gmail.com	+919676839916	₹200	6 yrs	rejected	Approve
3	Dr. Dr karthik	Dermatology	pandupooja1277@gmail.com	+919676839916	₹200	4 yrs	pending	Approve Reject
4	Dr. Dr Sai	Eye Specialist	2278a3101@gmail.com	9676839916	₹100	3 yrs	pending	Approve Reject
5	Dr. Dr jayalakshmi	Orthopedic	pandupooja1277@gmail.com	9121105789	₹250	5 yrs	approved	Reject
6	Dr. Rudra	General Physician	apoojasree515@gmail.com	9791846835	₹150	7 yrs	approved	Reject

Live Demo:

Project Demo URL: https://github.com/1112pooja/DocSpot-Seamless-Appointment-Booking-for-Health-.git

GitHub Repository: https://github.com/1112pooja/DocSpot-Seamless-Appointment-Booking-for-Health-.git

12. Known Issues

Issue ID	Module	Description	Severity	Status
BUG-01	Notifications	Appointment status notification not triggered in some edge cases when doctor updates status rapidly.	Medium	Open
BUG-02	Appointment Booking	Date picker does not restrict past dates on some mobile browsers, allowing past date selection.	High	In Progress

13. Future Enhancements

- **Add Razorpay or Stripe for online consultation fee collection at booking time.**Payment Gateway Integration:
- **Implement WebSocket-based chat and WebRTC video calls for telemedicine support.**Real-Time Chat & Video Consultation:
- **Integrate Nodemailer and Twilio to send appointment confirmations and reminders.**Email & SMS Notifications:
- **Build a recommendation engine based on patient symptoms, specialty, and doctor ratings.**AI Doctor Recommendation:
- **A real-time calendar system for doctors to manage availability and time slots dynamically.**Live Slot Management:
- **A secure, persistent module for patients to store prescriptions and medical history.**Patient Medical Records:
- **Charts and reports for appointment trends, doctor performance, and user activity.**Admin Analytics Dashboard:
- **Extend DocSpot to iOS and Android with a React Native mobile application.**React Native Mobile App:

14 CONCLUSION:

- DocSpot successfully delivers a comprehensive, role-based appointment booking platform that
- addressesthecorechallengesfacedbypatients,doctors,andhealthcareadministrators.Byleveraging the
- MERN stack, the application provides a secure, scalable, and user-friendly solution for managing
- healthcare appointments digitally.
- The system passed UAT with a 96.8% pass rate across 63 test cases, with only two minor open issues
- related to notifications and date validation. The platform demonstrates strong foundational architecture
- that can be extended with advanced features in future iterations.
- DocSpot represents a meaningful step toward digitizing the healthcare appointment experience,
- reducing friction for patients, improving efficiency for doctors, and providing administrators with robust
- oversight tools.